Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Nombre de la actividad curricular:	Fundamentos de Programación	
Código:	08381	
Créditos:	8 UAH / 5 SCT	
Carácter:	Obligatorio	
Prerrequisitos:	No tiene	
Tipo:	Curso / Laboratorio	
Horas cronológicas de dedicación	**Docencia directa:** 4,5	**Trabajo autónomo:** 3,5

Compendio de Ejercicios "tipo" por semana y casos de estudios por unidad

NOTA: Cada semana está compuesta por un ejercicio básico, uno intermedio y uno avanzado con objetivo y enunciado; fundados en Aprendizaje basado en problemas y alineados con el Resultado de Aprendizaje RA semanal. Al final cada unidad se presenta un estudio de caso fundado en aprendizaje basado en servicio para ser entregado en parejas equivalente al control de cada unidad.

UNIDAD I — Fundamentos de la Programación (Semanas 1–5)

Semana 1 — *RA: Explicar los conceptos del pensamiento computacional y reconocer sus componentes (abstracción, patrones, descomposición, algoritmos) mediante ejemplos simples.*

Básico – Clasificación de componentes CT (caso "kiosco del campus")
Objetivo: Reconocer y nombrar correctamente los cuatro componentes del pensamiento computacional.
Enunciado: Observa el proceso "Comprar desayuno en el kiosco del campus" (mirar menús, pagar, recibir vuelto).

1. Lista 6 pasos del proceso.
2. Identifica: descomposición (¿cómo dividirías la tarea?), patrones (¿qué se repite?), abstracción (¿qué detalles irrelevantes omites?), algoritmo (redacta 5–7 pasos ordenados).

Alineación con RA: Centrado en reconocer y explicar componentes con un ejemplo cercano (kiosco).

Intermedio – Reconocer patrones en vida universitaria
Objetivo: Distinguir patrones y proponer una abstracción.
Enunciado: Considera "Llegar a primera clase". Escribe 3 variantes (bus, metro, bicicleta). Marca patrones comunes y diferencias. Define una abstracción de alto nivel con 6–8 pasos que sirva a las tres variantes.
Alineación con RA: Énfasis en patrones y abstracción.

Avanzado – Diferenciar algoritmo vs. receta ambigua
Objetivo: Explicar qué vuelve ejecutable a un algoritmo.
Enunciado: Reescribe la instrucción "organiza tu mochila" como algoritmo determinístico (10–12 pasos, sin ambigüedades, decisiones claras). Luego marca qué parte es descomposición, qué patrones reutilizas y qué detalles omitiste (abstracción).
Alineación con RA: Explicación y ejemplificación directa de los conceptos.

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Semana 2 — *RA: Construir algoritmos básicos usando representaciones textuales y diagramáticas, justificando la secuencia lógica.*

Básico – Algoritmo textual del "pase diario RED"
Objetivo: Redactar un algoritmo paso a paso.
Enunciado: Escribe un algoritmo para "Cargar la tarjeta Bip!/TNE y verificar saldo suficiente para 2 viajes". Incluye validación de saldo y mensaje final.
Alineación con RA: Construcción textual + justificación de orden.
Intermedio – Diagrama de flujo "reserva de sala"
Objetivo: Traducir un proceso a diagrama de flujo.
Enunciado: Dibuja el flujo para "Reservar sala de estudio en biblioteca": ingreso solicitud, disponibilidad, alternativas y confirmación. Incluye 1 decisión con dos salidas y un bucle de búsqueda de horario.
Alineación con RA: Representación diagramática con lógica.

Avanzado – Refinamiento de algoritmo (pseudocódigo)
Objetivo: Especificar condiciones y salidas claras.
Enunciado: En pseudocódigo (no Python) define:

- Entradas: presupuesto diario ($), lista de precios colación.
- Proceso: seleccionar ítems hasta no superar presupuesto.
- Salida: lista elegida y saldo.

Alineación con RA: Construcción y justificación de la secuencia lógica.

Semana 3 — *RA: Analizar problemas identificando entradas, procesos y salidas (IPO) y representarlos con pseudocódigo inicial.*

Básico – Matriz IPO "consumo de agua en casa"
Objetivo: Separar claramente Entradas, Proceso, Salidas.
Enunciado: Para "Calcular consumo mensual de agua (m^3) y costo", llena:

- Entradas: lecturas medidor, tarifa, días.
- Proceso: diferencia lecturas, multiplicar por tarifa.
- Salidas: m^3, costo total.

Alineación con RA: Estructura IPO.

Intermedio – Pseudocódigo de movilidad
Objetivo: Plasmar IPO en pseudocódigo.
Enunciado: Escribe pseudocódigo para comparar emisiones de bus vs. auto compartido (parámetros como factores por km). Imprime alternativa más baja.
Alineación con RA: IPO + pseudocódigo.

Avanzado – Validación con tabla de trazas
Objetivo: Verificar lógica con desk-check.
Enunciado: Sobre tu pseudocódigo anterior, construye una tabla de trazas con 3 juegos de datos (cambia km y factores). ¿La salida coincide con lo esperado?
Alineación con RA: Análisis y validación del proceso.

Semana 4 — *RA: Diseñar soluciones estructuradas aplicando abstracción, modularidad y patrones algorítmicos.*

Básico – Selección de patrón "máximo/mínimo"
Objetivo: Elegir patrones algorítmicos.
Enunciado: Dada una lista de 8 temperaturas (papel), diseña un algoritmo modular con dos subprocedimientos: maximo() y minimo(). Entrega diagrama de módulos.
Alineación con RA: Modularidad + patrones.

Intermedio – División en módulos (agendamiento)
Objetivo: Diseñar arquitectura de alto nivel.
Enunciado: Para "Agendar ayudantías", propone módulos: entrada de datos, validación, conflicto de horarios, confirmación. Describe interfaz (entradas/salidas) de cada módulo.
Alineación con RA: Abstracción + modularidad.
Avanzado – Diseño con patrón "seleccionar-hasta-llenar presupuesto"
Objetivo: Diseñar solución reusable.
Enunciado: Propón un diseño con módulo de selección y módulo de evaluación de restricción (presupuesto). Muestra el flujo entre módulos y cómo podrías reutilizar el mismo patrón en otro contexto (tiempo en lugar de dinero).
Alineación con RA: Diseño estructurado y reusable.

Semana 5 — *RA: Integrar elementos de la Unidad I resolviendo problemas que combinen análisis, diseño y validación de algoritmos.*

Básico – Integración IPO + pseudocódigo (canasta básica)
Objetivo: Unir análisis y construcción.
Enunciado: Define IPO y pseudocódigo para seleccionar alimentos básicos cumpliendo monto mínimo de proteínas y tope de presupuesto.
Alineación con RA: Integración análisis + diseño.

Intermedio – Diagrama de flujo + trazas (bienestar digital)
Objetivo: Diseñar y validar.
Enunciado: Flujo para calcular tiempo total en apps y detectar la app más usada. Realiza trazas con 2 juegos de datos.
Alineación con RA: Diseño + validación.

Avanzado – Revisión por pares
Objetivo: Evaluar la corrección del algoritmo.
Enunciado: Intercambia tu pseudocódigo con otro equipo. Usa una lista de chequeo (entradas bien definidas, decisiones cubren todos los casos, salidas claras). Registra 3 mejoras.
Alineación con RA: Integración y mejora de calidad.

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Estudio de Caso ABpS — Unidad I

Unidad I: Fundamentos de la Programación

Propósito de aprendizaje: *Aplicar el pensamiento algorítmico y el pensamiento computacional para resolver problemas.*

Estudio de Caso ABpS — "Optimización del uso de agua en la Junta de Vecinos Los Alerces"

Objetivo del caso (alineado con el propósito de la unidad)

Que los estudiantes apliquen pensamiento computacional (descomposición, patrones, abstracción y algoritmos) para analizar un problema comunitario real y diseñar soluciones algorítmicas que permitan a la junta de vecinos comprender y reducir su consumo de agua.

Enunciado del caso

La Junta de Vecinos Los Alerces ha reportado un aumento del consumo de agua en áreas comunes y viviendas. Solicitan apoyo para:

1. Analizar el problema:
    - Identificar las fuentes principales de consumo,
    - Establecer patrones de uso según actividad (riego, lavado, consumo domiciliario),
    - Determinar variables relevantes.
2. Descomponer el problema en subproblemas manejables (riego, consumo familiar, fugas reportadas, etc.).
3. Crear representaciones:
    - Matriz IPO del problema,
    - Diagramas de flujo o pseudocódigo que representen la lógica de estimación del consumo,
    - Un algoritmo que calcule escenarios de reducción (10%, 20%, 30%).

El objetivo es producir un diseño algorítmico claro, sin necesidad aún de programar en Python (ya que inicia en la semana 6).

Producto esperado

- Documento entregable a la Junta de Vecinos que incluya:
    - Análisis del problema,
    - Patrones y descomposición,
    - Abstracciones pertinentes,
    - Un algoritmo paso a paso con opciones de simulación.

Servicio a la comunidad

El documento permitirá a la Junta de Vecinos planificar acciones de reducción de agua basadas en un modelo claro y comunicable, facilitando charlas internas y apoyo a campañas locales.

Cómo evidencia el propósito de la unidad

Los estudiantes deben aplicar pensamiento algorítmico y computacional para transformar un problema social real en un modelo algorítmico estructurado, demostrando dominio de todos los componentes de la unidad antes de programar.

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

UNIDAD II — Programación estructurada (Semanas 6–11)

Semana 6 — *RA: Implementar algoritmos en Python usando estructura básica de un programa, E/S y pruebas simples.*

Básico— Costo de agua por departamento (E/S y aritmética)

Objetivo: Implementar un programa que lea entradas concretas, realice cálculos directos y muestre salidas claras.
Enunciado: Pide al usuario los consumos de agua (m³) de tres departamentos y una tarifa ($/m³). Muestra el costo de cada departamento y el total.
Notas didácticas: Sin listas, sin bucles; solo variables, input() y print().
Sugerencia de código solución en Python:

```python
# Semana 6 - Básico: Costo de agua por departamento
# Entradas: m³ de 3 departamentos y tarifa $/m³
m3_d1 = float(input("m³ Depto 1: "))
m3_d2 = float(input("m³ Depto 2: "))
m3_d3 = float(input("m³ Depto 3: "))
tarifa = float(input("Tarifa ($/m³): "))

# Procesamiento: costo por depto y total
costo_d1 = m3_d1 * tarifa
costo_d2 = m3_d2 * tarifa
costo_d3 = m3_d3 * tarifa
total = costo_d1 + costo_d2 + costo_d3

# Salida: desglose y total
print("\n-- Resumen de costos --")
print("Depto 1: $", round(costo_d1, 0))
print("Depto 2: $", round(costo_d2, 0))
print("Depto 3: $", round(costo_d3, 0))
print("TOTAL:   $", round(total, 0))
```

Intermedio— Gasto en transporte RED (E/S, cálculo semanal y mensual)

Objetivo: Reforzar E/S y operaciones aritméticas con salidas amigables.
Enunciado: Solicita viajes por semana y tarifa (CLP por viaje). Calcula el gasto semanal y el gasto mensual asumiendo 4 semanas.
Sugerencia de código solución en Python:

```python
# Semana 6 - Intermedio: Gasto en transporte RED
viajes_semana = int(input("Viajes por semana: "))
tarifa = int(input("Tarifa por viaje (CLP): "))

gasto_semana = viajes_semana * tarifa
gasto_mes = gasto_semana * 4  # asumimos 4 semanas

print("\n-- Gasto estimado --")
print("Semanal: $", gasto_semana)
print("Mensual: $", gasto_mes)
```

Avanzado— Promedio y máximo de consumo eléctrico (sin bucles)
Objetivo: Practicar múltiples entradas, un cálculo agregado y una salida clara, manteniéndolo simple (sin listas ni bucles todavía).
Enunciado: Lee cuatro valores de kWh (de cuatro semanas) y muestra el promedio y el consumo máximo.
Sugerencia de código solución en Python:

```python
# Semana 6 - Avanzado (simple): Promedio y máximo de consumo
k1 = float(input("kWh semana 1: "))
k2 = float(input("kWh semana 2: "))
k3 = float(input("kWh semana 3: "))
k4 = float(input("kWh semana 4: "))

promedio = (k1 + k2 + k3 + k4) / 4
# Máximo sin usar listas: comparar por pasos
max1 = k1 if k1 >= k2 else k2
max2 = k3 if k3 >= k4 else k4
maximo = max1 if max1 >= max2 else max2

print("\n-- Consumo --")
print("Promedio mensual (kWh):", round(promedio, 1))
print("Máximo semanal (kWh):  ", maximo)
```

Semana 7 — RA: *Seleccionar y usar tipos de datos adecuados (numéricos, booleanos, cadenas) y conversiones (casting).*

Básico – Conversor UF a CLP parametrizado
Objetivo: Usar numéricos y casting.
Enunciado: Pide valor de la UF (float) y un monto en UF; imprime CLP.
Sugerencia de código solución en Python:

```python
# Semana 7 - Básico
valor_uf = float(input("Valor UF (CLP): "))
monto_uf = float(input("Monto en UF: "))
print(f"≈ ${valor_uf * monto_uf:,.0f} CLP")
```

Intermedio – Limpieza de texto de encuesta
Objetivo: Manipular strings.
Enunciado: Recibe un texto; normaliza (minúsculas, espacios) y cuenta “transporte”.
Sugerencia de código solución en Python:

```python
# Semana 7 - Intermedio (versión simple, sin split/join)
texto = input("Pega tu respuesta: ")

# Normalización básica: solo a minúsculas
t = texto.lower()

# Conteo de ocurrencias de la subcadena "transporte"
conteo = t.count("transporte")

print("Menciones de 'transporte':", conteo)
```

Avanzado – Booleanos y verificación de rango
Objetivo: Trabajar con booleanos y casteos.
Enunciado: Pide edad y si posee TNE (sí/no). Muestra si aplica tarifa estudiante
Sugerencia de código solución en Python:

```
# Semana 7 - Avanzado
edad = int(input("Edad: "))
tne = input("¿Posee TNE? (s/n): ").strip().lower() == "s"
aplica = (edad <= 28) and tne
print("Tarifa estudiante:", "Sí" if aplica else "No")
```

Semana 8 — RA: *Aplicar operadores aritméticos y lógicos y evaluar su impacto en el flujo.*

Básico – Cálculo calórico simple
Objetivo: Practicar operadores aritméticos.
Enunciado: Calcula balance calórico de una colación (parámetros simples).
Sugerencia de código solución en Python:

```
# Semana 8 - Básico
kcal = 2*250 + 120 - (30*4)
print("Balance (kcal):", kcal)
```

Intermedio – Descuentos condicionados
Objetivo: Componer expresiones con operadores lógicos.
Enunciado: Calcula pago aplicando 10% si total > $20.000 y 5% adicional si hay credencial.
Sugerencia de código solución en Python:

```
# Semana 8 - Intermedio
total = int(input("Total en librería: "))
cred = input("¿Credencial estudiante? (s/n): ").strip().lower() == "s"
pago = total * 0.90 if total > 20000 else total
pago = pago * 0.95 if cred else pago
print(f"A pagar: ${pago:,.0f}")
```

Avanzado – Expresión booleana compuesta
Objetivo: Razonamiento lógico.
Enunciado: Dado promedio y asistencia, aprueba si (prom ≥ 4.0 y asis ≥ 70%) **o** (prom ≥ 3.8, asis ≥ 80% y recuperación).
Sugerencia de código solución en Python:

```
# Semana 8 - Avanzado
prom = float(input("Promedio: "))
asis = int(input("Asistencia (%): "))
recup = input("¿Rindió recuperación? (s/n): ").strip().lower() == "s"
aprueba = (prom >= 4.0 and asis >= 70) or (prom >= 3.8 and asis >= 80 and recup)
print("Estado:", "Aprobado" if aprueba else "Reprobado")
```

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Semana 9 — RA: *Construir estructuras condicionales correctamente indentadas (if/elif/else) para decisiones múltiples.*

Básico – Clasificación PM2.5 (umbral didáctico)
Objetivo: Condicional simple y múltiple.
Enunciado: Clasifica calidad del aire con 3 categorías.
Sugerencia de código solución en Python:

```python
# Semana 9 - Básico
pm25 = float(input("PM2.5 promedio del día: "))
if pm25 <= 25: calidad = "Bueno"
elif pm25 <= 50: calidad = "Regular"
else: calidad = "Alerta"
print("Calidad:", calidad)
```

Intermedio – Tarifa RED según TNE y horario punta
Objetivo: Anidar condiciones.
Enunciado: Si TNE activa → tarifa estudiante; si no, tarifa general; si es punta, recargo 10%.
Sugerencia de código solución en Python:

```python
# Semana 9 - Intermedio
tne = input("¿TNE activa? (s/n): ").strip().lower() == "s"
punta = input("¿Horario punta? (s/n): ").strip().lower() == "s"
tarifa = 270 if tne else 800
if punta: tarifa *= 1.10
print(f"Tarifa aplicada: ${tarifa:.0f}")
```

Avanzado – Beca según tramos
Objetivo: Decisiones por rangos con prioridad.
Enunciado: Asigna “Beca A, B, C o Sin beca” según promedio y vulnerabilidad (0–100).
Sugerencia de código solución en Python:

```python
# Semana 9 - Avanzado
prom = float(input("Promedio: "))
vuln = int(input("Índice de vulnerabilidad (0-100): "))
if prom >= 6.0 and vuln >= 60: beca = "A"
elif prom >= 5.5 and vuln >= 40: beca = "B"
elif prom >= 5.0 and vuln >= 20: beca = "C"
else: beca = "Sin beca"
print("Resultado:", beca)
```

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Semana 10 — RA: *Integrar elementos de la Unidad II en programas con control de flujo completo.*

Básico – Simulador de notas
Objetivo: Integrar entrada, condicionales y promedio.
Enunciado: 3 controles (20%), 2 labs (30%), certamen (50%).
Sugerencia de código solución en Python:

```python
# Semana 11 - Básico
controles = [float(input(f"C{i+1}: ")) for i in range(3)]
labs = [float(input(f"L{i+1}: ")) for i in range(2)]
cert = float(input("Certamen: "))
final = sum(controles)/3*0.2 + sum(labs)/2*0.3 + cert*0.5
print(f"Nota final: {final:.1f}")
```

Intermedio – Presupuesto Fiestas Patrias en sede
Objetivo: Integrar selección con tope de presupuesto.
Enunciado: Lista de ítems (nombre, costo, esencial). Selecciona primero esenciales y luego opcionales sin superar presupuesto.
Sugerencia de código solución en Python:

```python
# Semana 11 - Intermedio
propuestos = [("Empanadas",30000,True),("Bebidas",12000,True),
              ("Decoración",8000,False),("Música",5000,False)]
presupuesto = int(input("Presupuesto: "))
carro, total = [], 0
for n,c,e in propuestos:
    if e and total + c <= presupuesto:
        carro.append(n); total += c
for n,c,e in propuestos:
    if not e and total + c <= presupuesto:
        carro.append(n); total += c
print("Selección:", carro, "| Total:", total)
```

Avanzado – Reporte E/S y bucles (RED mensual)
Objetivo: Integrar E/S, bucles y decisiones.
Enunciado: Lee viajes por semana (4 valores), tarifa y TNE. Calcula gasto total y promedio; si TNE, aplica tarifa estudiante.
Sugerencia de código solución en Python:

```python
# Semana 11 - Avanzado
viajes = [int(input(f"Viajes semana {i+1}: ")) for i in range(4)]
tne = input("¿TNE activa? (s/n): ").strip().lower() == "s"
tarifa = 270 if tne else 800
gastos = [v*tarifa for v in viajes]
print("Gasto total:", sum(gastos), " | Promedio/sem:", sum(gastos)/4)
```

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Semana 11 — RA: *Diseñar e implementar ciclos (for/while), contadores, acumuladores y rangos.*

Básico – Acumulador de ahorro
Objetivo: while + acumulador.
Enunciado: Simula ahorro de $20.000/mes con interés 0,2% hasta meta.
Sugerencia de código solución en Python:

```python
# Semana 10 - Básico
meta, saldo, mes = 500000, 0, 0
while saldo < meta:
    mes += 1
    saldo += 20000
    saldo *= 1.002
print(f"Meses: {mes}, Saldo: ${saldo:,.0f}")
```

Intermedio – Conteo de palabra en comunicado
Objetivo: Recorrer texto con for.
Enunciado: Cuenta “sustentabilidad” (insensible a mayúsculas).
Sugerencia de código solución en Python:

```python
# Semana 10 - Intermedio
texto = input("Texto: ").lower()
conteo = 0
for palabra in texto.split():
    if "sustentabilidad" in palabra:
        conteo += 1
print("Apariciones:", conteo)
```

Avanzado – Suma de serie con rango
Objetivo: Rango y acumulación.
Enunciado: Calcula ∑_{i=1..n} i*(i+1).
Sugerencia de código solución en Python:

```python
# Semana 10 - Avanzado
n = int(input("n: "))
s = 0
for i in range(1, n+1):
    s += i*(i+1)
print("Suma:", s)
```

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Estudio de Caso ABpS — Unidad II

Unidad II: Programación Estructurada

Propósito de aprendizaje: *Aplicar los elementos de programación estructurada, como tipos de datos, variables, aritmética, funciones y estructuras de control.*

Estudio de Caso ABpS — "Sistema de Gestión de Préstamos para la Biblioteca Municipal de Nueva Aurora"

Objetivo del caso (alineado con el propósito de la unidad)

Que los estudiantes implementen un prototipo funcional en Python utilizando tipos de datos, variables, condicionales, ciclos y funciones, para resolver una necesidad real de la biblioteca municipal.

Enunciado del caso

La Biblioteca Municipal necesita digitalizar su control de préstamos, ya que el registro manual causa pérdidas de información y retrasos.

Se pide a los estudiantes desarrollar un programa en Python que permita:

1. Registrar usuarios (nombre, RUT, categoría: niño, adulto, adulto mayor).
2. Registrar libros (título, autor, categoría).
3. Registrar préstamos, validar disponibilidad y calcular fechas de devolución.
4. Calcular multas utilizando aritmética simple y estructuras de control:
    - Se sugiere un valor por día de atraso definido para el ejercicio.
5. Crear funciones para modularidad:
    - registrar_usuario(),
    - registrar_libro(),
    - prestar_libro(),
    - calcular_multa(), etc.
6. Generar un reporte semanal de préstamos activos.

El programa debe construirse totalmente con programación estructurada, sin uso de estructuras de datos avanzadas (propias de la unidad III).

Producto esperado

- Un prototipo ejecutable en Python, interactivo vía consola.
- Manual de uso para funcionarios de la Biblioteca.

Servicio a la comunidad

La Biblioteca podrá evaluar la adopción del prototipo como apoyo al registro real y potenciar el acceso a libros para escolares y adultos.

Cómo evidencia el propósito de la unidad

Cada parte del caso obliga a usar:

- Variables para almacenar información,
- Tipos de datos adecuados,
- Aritmética para multas o tiempos,
- Condicionales para validaciones,
- Ciclos para listados,
- Funciones para modularidad.

Es una aplicación directa de los elementos de programación estructurada.

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

UNIDAD III — Estructuras de datos básicas y funciones (Semanas 12–16)

Semana 12 — RA: *Implementar y manipular listas y tuplas (almacenar, recorrer y modificar).*

Básico – Métricas de laboratorio
Objetivo: Recorrer lista y calcular métricas.
Enunciado: Con temperaturas (lista), imprime min, max y promedio.
Sugerencia de código solución en Python:

```python
# Semana 12 - Básico
temps = [20.1, 21.3, 22.0, 22.5, 21.8, 20.7, 19.9]
print("Min:", min(temps), "Max:", max(temps), "Prom:", sum(temps)/len(temps))
```

Intermedio – Próximo bus (tupla ordenada)
Objetivo: Indexación y búsqueda.
Enunciado: Con una tupla de horarios (min desde 00:00), entrega el próximo ≥ consulta.
Sugerencia de código solución en Python:

```python
# Semana 12 - Intermedio
horarios = (480, 540, 600, 660, 720)  # 8:00..12:00
consulta = int(input("Minutos desde 00:00: "))
prox = next((h for h in horarios if h >= consulta), None)
print("Próximo:", prox if prox else "Mañana")
```

Avanzado – Operaciones de lista (filtrado y mapeo)
Objetivo: Modificar y derivar listas.
Enunciado: Dada lista de gastos diarios filtra > $3.000 y ajusta 5% de inflación.
Sugerencia de código solución en Python:

```python
# Semana 12 - Avanzado
gastos = [2800, 3100, 2900, 3200, 2500, 4000]
altos = [g for g in gastos if g > 3000]
ajustados = [round(g*1.05) for g in altos]
print("Altos:", altos, "| Ajustados:", ajustados)
```

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Semana 13 — RA: *Emplear diccionarios y conjuntos para resolver problemas con relaciones clave–valor y pertenencia.*

Básico – Total de canasta con diccionarios
Objetivo: Totalizar con claves.
Enunciado: Con precios y cantidades, calcula total.
Sugerencia de código solución en Python:

```python
# Semana 13 - Básico
precios = {"arroz":1400,"fideos":1200,"aceite":2500,"atún":1300,"leche":900}
cant = {"arroz":2,"fideos":3,"aceite":1,"atún":4,"leche":6}
total = sum(precios[p]*cant[p] for p in cant)
print("Total:", total)
```

Intermedio – Top 3 productos más caros
Objetivo: Ordenar ítems por valor.
Enunciado: Muestra top-3 por precio y su contribución al total.
Sugerencia de código solución en Python:

```python
# Semana 13 - Intermedio
top3 = sorted(precios.items(), key=lambda x: x[1], reverse=True)[:3]
contrib = sum(precios[n]*cant.get(n,0) for n,_ in top3)/total*100
print("Top3:", top3, "| % del total:", f"{contrib:.1f}%")
```

Avanzado – Conjuntos de asistencia
Objetivo: Operaciones de conjuntos.
Enunciado: Conjuntos A (ayudantía) y B (lab). Entrega A∩B, A−B y B−A.
Sugerencia de código solución en Python:

```python
# Semana 13 - Avanzado
A = {"Ana","Benja","Cami","Diego"}
B = {"Cami","Ema","Fran","Diego"}
print("Ambos:", A & B)
print("Solo A:", A - B)
print("Solo B:", B - A)
```

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Semana 14 — RA: *Definir funciones bien estructuradas aplicando modularidad, reutilización y separación de responsabilidades.*

Básico – Funciones de conversión (UF/US$ a CLP)
Objetivo: Definir y reutilizar funciones puras.
Enunciado: Crea funciones simples que conviertan montos desde UF y desde dólares a CLP usando valores entregados por el usuario.
Sugerencia de código solución en Python:

```python
# Semana 14 - Básico
def uf_a_clp(valor_uf, monto_uf):
    return valor_uf * monto_uf

def usd_a_clp(valor_usd, monto_usd):
    return valor_usd * monto_usd

print(uf_a_clp(37000, 1.5))
print(usd_a_clp(900, 200))
```

Intermedio – IMC con separación de responsabilidades
Objetivo: Módulos pequeños y testeables.
Enunciado: Implementa una función para calcular el IMC y otra para clasificarlo en categorías.
Sugerencia de código solución en Python:

```python
# Semana 14 - Intermedio
def imc(peso, altura):
    return peso / (altura**2)

def categoria_imc(v):
    if v < 18.5: return "Bajo peso"
    if v < 25:   return "Normal"
    if v < 30:   return "Sobrepeso"
    return "Obesidad"

valor = imc(68, 1.70)
print(valor, categoria_imc(valor))
```

Avanzado – Canalización de funciones (pipeline)
Objetivo: Componer funciones reutilizables.
Enunciado: Dada una lista de gastos, crea funciones filtra_mayores(th), ajusta_inflacion(p), promedio(lista) y compón el pipeline.
Sugerencia de código solución en Python:

```python
# Semana 14 - Avanzado
def filtra_mayores(valores, th):
    return [v for v in valores if v > th]

def ajusta_inflacion(valores, p):
    return [round(v*(1+p)) for v in valores]

def promedio(valores):
    return sum(valores)/len(valores) if valores else 0

g = [2800, 3100, 2900, 3200, 2500, 4000]
res = promedio(ajusta_inflacion(filtra_mayores(g, 3000), 0.05))
print("Promedio ajustado:", round(res,1))
```

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Semana 15 — RA: *Aplicar paso de parámetros y retorno; distinguir variables locales y globales.*

Básico – Costo de energía con parámetros
Objetivo: Practicar paso de parámetros y retorno.
Enunciado: Implementa una función que reciba kWh, tarifa base y cargo fijo, y devuelva el costo total.
Sugerencia de código solución en Python:

```python
# Semana 15 - Básico
def costo_energia(kwh, tarifa_base, cargo_fijo):
    return kwh * tarifa_base + cargo_fijo

print(costo_energia(120, 150, 2500))

```

Intermedio – Alcance de variables (local vs global)
Objetivo: Distinguir variables locales y globales.
Enunciado: Crea una función que calcule un costo usando una variable local con el mismo nombre que una global y compara los resultados.
Sugerencia de código solución en Python:

```python
# Semana 15 - Intermedio
tarifa_base = 150  # global

def calcula_costo(kwh, cargo_fijo):
    tarifa_base = 160  # local
    return kwh * tarifa_base + cargo_fijo

print("Global:", tarifa_base)
print("Costo calculado:", calcula_costo(100, 2500))
```

Avanzado – Validador de RUT como función reusable
Objetivo: Usar parámetros, retorno y lógica modular.
Enunciado: Implementa una función que reciba un RUT con formato y valide el dígito verificador.
Sugerencia de código solución en Python:

```python
# Semana 15 - Avanzado
def valida_rut(rut: str) -> bool:
    limpio = rut.replace(".", "").replace("-", "").lower()
    cuerpo, dv = limpio[:-1], limpio[-1]
    if not cuerpo.isdigit():
        return False

    suma, mult = 0, 2
    for d in reversed(cuerpo):
        suma += int(d) * mult
        mult = 2 if mult == 7 else mult + 1

    resto = 11 - (suma % 11)
    dv_calc = "0" if resto == 11 else "k" if resto == 10 else str(resto)
    return dv_calc == dv

print(valida_rut("12.345.678-5"))
```

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Semana 16 — RA: *Integrar elementos de la Unidad III en programas modulares con estructuras de datos y funciones interrelacionadas.*

Básico – Totales por material (reciclaje)
Objetivo: Combinar listas/diccionarios.
Enunciado: Sumar kg por material desde una lista de registros.
Sugerencia de código solución en Python:

```python
# Semana 16 - Básico
registros = [
    {"fecha":"2026-03-01","material":"papel","kg":5.3},
    {"fecha":"2026-03-01","material":"vidrio","kg":7.1},
    {"fecha":"2026-03-02","material":"papel","kg":2.0},
]
totales = {}
for r in registros:
    totales[r["material"]] = totales.get(r["material"], 0) + r["kg"]
print("Totales:", totales)
```

Intermedio – Filtro por ramo con función
Objetivo: Usar funciones y listas de diccionarios para consultar y devolver subconjuntos.
Enunciado: Usa funciones + listas de diccionarios para filtrar tutorías por ramo.
Sugerencia de código solución en Python:

```python
# Semana 16 - Intermedio
tutorias = [
    {"ramo":"FP", "tutor":"Ana","dia":"Lun","hora":"10:00"},
    {"ramo":"Cálculo", "tutor":"Benja","dia":"Mar","hora":"12:00"},
    {"ramo":"FP", "tutor":"Cami","dia":"Jue","hora":"09:00"},
]
def filtra_por_ramo(lista, ramo):
    return [t for t in lista if t["ramo"] == ramo]
print(filtra_por_ramo(tutorias, "FP"))
```

Avanzado – Mini-sistema modular (menú simple)
Objetivo: Integrar funciones, listas/diccionarios y control de flujo en un programa modular.
Enunciado: Implementa un menú con 3 opciones: (1) agregar registro {fecha, material, kg}, (2) ver totales por material, (3) salir.
Sugerencia de código solución en Python:

```python
# Semana 16 - Avanzado (versión con bandera)

registros = []

def agregar(fecha, material, kg):
    registros.append({"fecha": fecha, "material": material, "kg": kg})

def totales_por_material():
    totales = {}
    for r in registros:
        mat = r["material"]
        totales[mat] = totales.get(mat, 0) + r["kg"]
    return totales

# Variable testigo / bandera
salida = False

while not salida:
    print("\n1) Agregar registro")
    print("2) Ver totales por material")
    print("3) Salir")

    opcion = input("> ").strip()

    if opcion == "1":
        fecha = input("Fecha (YYYY-MM-DD): ")
        material = input("Material: ")
        kg = float(input("kg: "))
        agregar(fecha, material, kg)
        print("Registro agregado.")

    elif opcion == "2":
        print("Totales:", totales_por_material())

    elif opcion == "3":
        print("Saliendo del sistema...")
        salida = True   # actualización de la variable bandera

    else:
        print("Opción no válida. Intente nuevamente.")
```

Facultad de Ingeniería / Ingeniería Civil Plan Común Ingeniería Civil en Informática/Ingeniería Civil Industrial

Estudio de Caso ABpS — Unidad III

Unidad III: Estructuras de datos básicas
Propósito de aprendizaje: *Aplicar estructuras de datos básicas para mejorar la calidad del código.*

Estudio de Caso ABpS — "Sistema de Asistencia y Progreso Académico para un Colegio Municipal"

Objetivo del caso (alineado con el propósito de la unidad)

Que los estudiantes diseñen e implementen un módulo en Python que utilice listas, tuplas, diccionarios y conjuntos para gestionar información académica de forma eficiente y clara, atendiendo una necesidad comunitaria real.

Enunciado del caso

El Colegio Municipal Los Boldos solicita apoyo para mejorar su seguimiento de asistencia y rendimiento en talleres STEM.
Los estudiantes deben construir un módulo Python que permita:

1. Registrar estudiantes (diccionarios con RUT, nombre, curso).
2. Llevar asistencia por fecha mediante:
    - Conjuntos (presentes, ausentes), o
    - Listas/diccionarios indexados por fecha.
3. Registrar notas, usando estructuras de datos adecuadas:
    - p.ej. diccionario {rut: [n1, n2, n3]}
4. Calcular promedios y generar un reporte final por curso.
5. Usar funciones que manipulen estas estructuras de datos.
6. Demostrar mejoras de calidad de código:
    - Legibilidad,
    - Modificabilidad,
    - Reutilización.

Producto esperado

- Un módulo Python que registre asistencia y notas.
- Un informe final a docentes describiendo la estructura de datos usada, su justificación y cómo mejora la mantenibilidad.

Servicio a la comunidad

El caso entrega al colegio un prototipo que puede servir de base para sistemas más formales, mejorando la trazabilidad del aprendizaje estudiantil.

Cómo evidencia el propósito de la unidad

Los estudiantes deben aplicar estructuras de datos básicas (listas, tuplas, diccionarios, conjuntos) para resolver necesidades reales.
Esto demuestra la comprensión de cómo mejoran la calidad del código en términos de estructura, claridad y eficiencia.